SciForge: A Multimodal Research-Native AI Workbench for Scientific Discovery

Author Names[1*]

[1*]Institute, University, City, Country.

Corresponding author(s). E-mail(s): author@example.com;

Abstract

Scientific work increasingly spans heterogeneous artifacts—papers, code, datasets, scientific file formats, model outputs, figures, manuscripts, and team decisions—yet general-purpose AI assistants rarely preserve these objects as a coherent, auditable research state. We present SciForge, a multimodal research-native AI workbench that reserves the graphical interface for human judgment while search, parsing, model routing, workflow execution, plotting, writing, and presentation generation run as modular agent-accessible services. SciForge is built around three pillars: (i) *translate-then-reason* for **multimodal** input, routing scientific objects through domain translators before the agent reasons; (ii) *evidence governance* for **auditable** traceability, linking claims to provenance chains and audit findings; and (iii) *goal-scoped scientific decision governance* with review gates and shared review surfaces. The system combines a thin interaction layer, contextual research capability patterns, an Agent Runtime and Workflow Engine, an Evidence-DAG audit sidecar, a Scientific Model Router, and local scientific memory backed by reusable worker services. The system is open-source and available at https://github.com/AGI4Sci/SciForge.

Keywords: SciForge, AI workbench, scientific discovery, agentic workflow, evidence-aware AI, multimodal science, research automation

1 Introduction

1.1 Background and Motivation

Scientific discovery is becoming increasingly computational, multimodal, and collaborative. A single research project may move among scholarly papers, experimental protocols, command-line tools, notebooks, gene and variant files, protein sequences,

three-dimensional structures, molecular records, single-cell matrices, generated figures, manuscript drafts, slide decks, and group decisions. These objects are not independent files. A manuscript claim may depend on a raw dataset, a preprocessing script, a model run, a visualization parameter, a literature passage, and a later human decision about whether the evidence is strong enough to use. As a result, the central bottleneck is no longer only access to models or tools, but the lack of a persistent environment that can organize scientific objects, agent actions, human decisions, and evidence traces as one coherent research state.

Recent AI systems have demonstrated impressive capabilities for literature search, hypothesis generation, coding, experimentation, and scientific writing [8–11, 16]. At the same time, desktop and terminal-capable scientific workbenches show the importance of keeping execution close to local files, remote servers, and HPC environments [3, 5, 12, 15, 17]. However, most existing systems still emphasize either task-specific intelligence, conversational assistance, or local execution surfaces. They rarely treat the full research process as an **auditable**, **multimodal**, and **collaborative** workflow in which scientific data, agent reasoning, generated artifacts, and manuscript claims remain connected over time.

This gap matters because scientific AI agents must satisfy three demands that are absent from general-purpose assistants. First, **multimodal** inputs are the norm: scientific files arrive in native representations poorly suited to direct prompting, including FASTA sequences, PDB or mmCIF structures, SMILES strings, variant files, and cell-omics tables. Second, **auditable** evidence is non-negotiable: researchers need to know which data, script, model, parameter, paper, or expert translation supported a conclusion; whether a figure matches the claim it is used to support; and when a proposed action requires human or principal-investigator approval. Third, **collaborative** governance is essential: scientific work involves team decisions, goal-scoped approvals, cross-session handoffs, and shared review of claims and evidence. A stateless chat transcript addresses none of these demands.

We present *SciForge*, a multimodal research-native AI workbench for scientific discovery. SciForge is designed as a local-first operating environment for research rather than a wrapper around a single model. Its architecture organizes the following interconnected functional systems: (1) **User Interaction & Collaboration** surfaces for desktop execution, group coordination, and mobile supervision; (2) **Research Capability Patterns** that expose literature triage, ideation, manuscript construction, artifact review, and presentation refinement inside one workbench; (3) a **Core Engine** combining agent runtime, workflow automation, goal-centered memory, and evidence governance; (4) a **Scientific Model Router** for modality detection and domain-expert translation; and (5) an **Infrastructure** layer providing local scientific memory, modular workers, scientific connectors, and team synchronization.

SciForge's core scientific contribution is organized around three pillars, each addressing a distinct gap in current research AI systems. **(i) Native scientific object processing with translate-then-reason:** end-to-end file ingress exposes three classes—protein sequences (.faa; .fasta/.fa with per-record conservative content classification), protein structures (.pdb/.cif/.mmcif), and molecules (.smi/.smiles)—routed through domain-expert translators (Esm2Text, Prot2Text, and BioT5) that

return structured expert observations before the main agent reasons. C2S is a worker API contract, not an end-to-end file-ingress route; .mol/.sdf are not auto-translated. VCF, BED, GFF, and MGF are unsupported and fail closed; raw content is not routed to the general reasoner. The translator output is an evidence candidate, not a verified scientific fact; the human remains responsible for judgment. **(ii) Artifact- and run-grounded evidence governance:** the Evidence DAG schema can capture ArtifactVersion, SourceAnchor, and run lineage when the runtime supplies structured evidenceLineage; software version, parameters, environment, log/output, and random seed are explicit provenance breakpoints that the schema represents but does not infer from prose. Immutable thread-level Evidence Snapshots are committed at user-designated breakpoints; provenance is not automatically complete per claim. A read-only audit sidecar inspects these snapshots without blocking agent exploration. **(iii) Goal-scoped scientific decision governance:** a Project DAG composes committed Evidence Snapshots with ReviewItem, DecisionEvent, an execution policy (autonomous, checkpointed, or supervised per goal), and candidate/certified release semantics. Audit findings are fail-open by default; blocking applies only to product actions explicitly integrated with the release API.

The local-first workspace, multi-provider Model Router, GUI judgment surfaces, and IM-based team coordination serve as the supporting architecture that makes these three pillars operational.

1.2 Contributions

This report makes the following contributions, organized around three pillars of scientific uniqueness:

- **Native Scientific Object Processing with Translate-then-Reason.** End-to-end file ingress exposes three classes: protein (.faa; .fasta/.fa with per-record conservative content classification), protein_structure (.pdb/.cif/.mmcif), and molecule (.smi/.smiles). Currently supported domain translators—Esm2Text for protein, Prot2Text for protein structures, BioT5 for small molecules—convert these native encodings into structured expert observations. C2S is a worker API contract, not an end-to-end file ingress route. The translator output is an evidence candidate whose provenance (modality, expert id, input reference, summary) is attached; the main agent then reasons over inspectable observations rather than opaque raw formats. VCF, BED, GFF, and MGF are unsupported and fail closed; raw content is not routed to the general reasoner. Literature is handled through search, PDF anchoring, and source attribution, not as a scientific modality translator.
- **Artifact- and Run-Grounded Evidence Governance.** The Evidence DAG schema can capture ArtifactVersion, SourceAnchor, and run lineage when the runtime supplies structured evidenceLineage; software version, parameters, environment, log/output, and random seed are explicit provenance breakpoints that the schema represents but does not infer from prose. Immutable thread-level Evidence Snapshots are committed at user-designated breakpoints. A read-only audit sidecar inspects snapshots and produces Risk Digests without blocking agent exploration. Audit is fail-open by default; blocking applies only where a product action

is explicitly integrated with the Project DAG release API. This separates evidence inspection from agent execution and provides explicit boundaries between what is automatically recorded and what requires human judgment.

- **Goal-Scoped Scientific Decision Governance.** A Project DAG composes committed Evidence Snapshots with ReviewItem, DecisionEvent, execution policy (autonomous, checkpointed, or supervised per goal), and candidate/certified release semantics. Curated subsets of Evidence Snapshots produce Project Snapshots suitable for external audit or publication. The GUI provides review and release-decision surfaces; IM-based channels (Zulip, Discord, WeChat, Feishu) currently support remote task submission, status supervision, and coordination rather than end-to-end Project DAG approval. Goal-centered memory stores scoped free-text records (scope, tags, confidence, timestamps, context); typed goals and decision events belong to the Project DAG.

The local-first workspace, multi-provider Scientific Model Router, thin GUI judgment surfaces, and IM-based team coordination serve as the supporting architecture that makes these three scientific pillars operational. SciForge treats each scientific interaction—literature triage, hypothesis refinement, manuscript drafting, artifact inspection, and presentation—as a first-class research task with explicit evidence trails, audit findings, and governance checkpoints. The SciForge codebase, including the Evidence-DAG audit worker, lightweight decision-record contracts, agentic skills, scientific model router, and the scenario-01 research-sprint gallery, is open source and available at https://github.com/AGI4Sci/SciForge.

2 Related Work

2.1 GUI-Based Scientific Workbenches

GUI-based scientific workbenches, including desktop and web applications, make AI assistance accessible through human-facing workspaces. Claude Science is the closest broad commercial reference: it is described as an AI workbench for scientists with desktop, local, SSH, and HPC-facing execution, scientific skills, specialist agents, generated artifacts, and reviewer agents for citations and calculations [3]. ClaudePrism focuses on the manuscript side of the workflow, combining a native desktop environment with local LaTeX compilation, Python environments, Claude Code sessions, and scientific writing skills [5]. Web tools such as Elicit and Consensus provide polished interfaces for literature search, synthesis, and research-agent style review [6, 7]. These systems demonstrate the importance of usable research interfaces, but they are usually centered on one model family, one workflow domain, or one user session. SciForge instead treats the GUI as a thin layer for human judgment over a local, modular, multi-agent research substrate.

2.2 Terminal Workbenches

Terminal-oriented workbenches prioritize execution close to local files, remote servers, and HPC environments. OmicOS provides a Rust `omicos-core` binary that can run

as a local daemon serving a browser UI or as `omicos cli`, with analysis code running in a shared IPython kernel and raw data remaining local [12–14]. OmicsClaw follows a local-first multi-omics design with CLI, TUI, desktop, web backend, and SSH remote execution surfaces over a shared agent loop [15]. Operon targets bioinformatics workflows across desktop and remote cluster environments, using persistent terminal sessions so jobs execute where data and schedulers live [17]. These systems are strong at local and remote execution, but they generally expose tools and workflows rather than a unified layer for multimodal scientific routing, research decision surfaces, team supervision, and manuscript-to-artifact continuity.

2.3 API and Research Agents

API-based and research-prototype agents explore more autonomous scientific reasoning. FutureHouse exposes agents for literature search, deep review, precedent search, and chemistry planning through a web/API platform [9]. Google's AI Co-Scientist studies multi-agent hypothesis generation, debate, and ranking for biomedical research [10]. The AI Scientist and Agent Laboratory automate parts of the machine-learning research loop, including ideation, coding, experimentation, paper drafting, and review [11, 16]. Literature systems such as PaperQA2, OpenScholar, and ScholarQA focus on retrieval-augmented scientific question answering and literature synthesis [1, 4, 8]. These systems demonstrate the power of specialized agents, but they are less focused on being a persistent local workbench where data, code, figures, decisions, and generated artifacts remain in one workspace.

2.4 Comparison and Positioning

While each of the three system categories addresses a distinct facet of the AI-assisted research life cycle, SciForge is designed to unify them within a single workbench. The comparison spans four key dimensions: execution, model governance, evidence quality, and research continuity.

Execution architecture. GUI workbenches such as Claude Science and ClaudePrism deliver polished interactive experiences but are typically constrained to one model provider and one local session. Terminal workbenches—OmicOS with its hybrid daemon plus browser model, OmicsClaw with multi-surface CLIs, and Operon with persistent terminal sessions—excel at running code where data and schedulers reside, yet they expose tools and pipelines rather than a unified research workbench. SciForge bridges these worlds by providing a GUI for human judgment while keeping a local-first workspace where execution can span desktop, SSH, and HPC environments.

Model governance. The reviewed public materials commonly emphasize a single provider family or manual switching between separate tools. SciForge's Scientific Model Router combines multi-provider routing with a deliberately narrow translate-then-reason file-ingress path for protein sequences, protein structures, and molecules. C2S remains a worker API contract rather than an end-to-end file route, and unsupported protected formats fail closed. Literature is handled through search, PDF anchoring, and source attribution, not as a scientific modality translator. We did

not find this same integrated combination documented as a first-class feature in the reviewed categories.

Evidence quality and governance. Research agents such as FutureHouse, AI Co-Scientist, and the AI Scientist produce outputs from variable-quality evidence; in the publicly documented materials we reviewed, we did not observe the same integrated combination of claim–evidence audit trail, PROV-JSON provenance, and quality gates in a single research workbench. SciForge threads claims through an Evidence DAG backed by PROV-JSON provenance, audit runs, risk digests, and explicit quality gates. This governance layer is designed to make agent outputs inspectable and auditable.

Research continuity. Many reviewed systems foreground per-session interaction. SciForge maintains goal-centered research memory as scoped free-text records (scope, tags, confidence, timestamps, context) across sessions; typed goals and decision events belong to the Project DAG, enabling long-running multi-session scientific workflows rather than only single-turn conversations. The combination of research memory, workflow engine, and evidence governance creates a persistent research workspace whose integrated design—tying thread-level Evidence Snapshots to goal-scoped Project Snapshots with audit and release semantics—we did not find documented as a first-class, integrated feature in the public materials of the reviewed GUI, terminal, and API-agent systems.

Table 1 summarizes these differences at the category level, while the detailed capability differentiation in Appendix A1 provides a feature-level comparison against common approaches.

Table 1 High-level comparison of scientific AI system categories. A checkmark indicates first-class or documented support as a primary design goal in the publicly reviewed materials. Absence of a checkmark does not prove absence of a feature; it indicates we did not find documented first-class support in the reviewed materials.

Dimension	GUI WB	Terminal WB	API agent	SciForge
Human-facing UI	✓		✓	✓
Local/HPC execution		✓		✓
Scientific tools	✓	✓	✓	✓
Local-first workspace		✓		✓
Multi-provider routing				✓
Scientific multimodal routing				✓
Human decision surfaces	✓			✓
Team workspace sync				✓
Modular worker services		✓		✓

SciForge combines these categories rather than competing with only one of them. It keeps the usability of GUI workbenches, the locality and composability of terminal systems, and the task power of scientific agents, while adding fail-closed scientific-object handling, content-addressed artifacts with structured selection and provenance, specialist domain translation, run-grounded evidence governance with immutable

snapshots and an audit sidecar, and goal-scoped decision governance with explicit release semantics.

3 System Architecture

3.1 Overall Design Philosophy

SciForge is organized as a layered research operating environment rather than a single-purpose chat interface. **Layer 1—User Interaction & Collaboration**—provides desktop execution, and group/mobile surfaces via Zulip, Discord, WeChat, and Feishu integrations for team coordination and lightweight supervision. **Layer 2—Research Capability Patterns**—unifies literature triage, hypothesis exploration, protocol design, paper reproduction, scholarly writing, visual review, and presentation refinement as contextual decision patterns in a single workbench, underpinned by thread-scoped Evidence DAGs and a goal-scoped Project DAG. **Layer 3—Core Engine**—combines the Agent Runtime, Automation Engine, Goal-Centered Research Memory, and Governance Engine for long-running, evidence-aware execution. **Layer 4—Scientific Model Router**—detects modality, routes inputs to domain experts, and returns provenance-annotated evidence to the reasoning agent. **Layer 5—Infrastructure**—provides local scientific memory, modular worker capabilities, scientific connectors, and team workspace synchronization.

SciForge follows three implementation principles that distinguish it from feature-heavy desktop software:

1. **Thin GUI as human-judgment surface.** The graphical interface is justified only when interaction, inspection, annotation, comparison, or approval requires human judgment; otherwise the capability belongs to the background agent runtime.
2. **Non-interactive capabilities delegated via modular services.** Search, scientific parsing, plotting, literature monitoring, model routing, workflow execution, and presentation generation run as MCP-compatible tools, HTTP services, or worker processes rather than tightly coupled UI code [2]. This keeps the application close to a thin interaction shell while tools evolve independently.
3. **Local-first, evidence-aware orchestration.** User data, project files, model traces, artifacts, and evidence graphs remain anchored to the research workspace. Remote collaboration and mobile access are control channels over the local workbench, not a replacement for local scientific state.

Figure 1 summarizes SciForge as a layered workbench—requests flow from collaboration surfaces through the Agent Runtime and contextual research capability patterns, while evidence traces, audit findings, and human decisions are maintained by background services.

3.2 User Interaction and Collaboration (Layer 1)

The top layer provides three interaction surfaces unified by the same runtime semantics. **Desktop execution** keeps the agent close to local files, servers, and HPC environments, enabling direct manipulation of research data without network latency

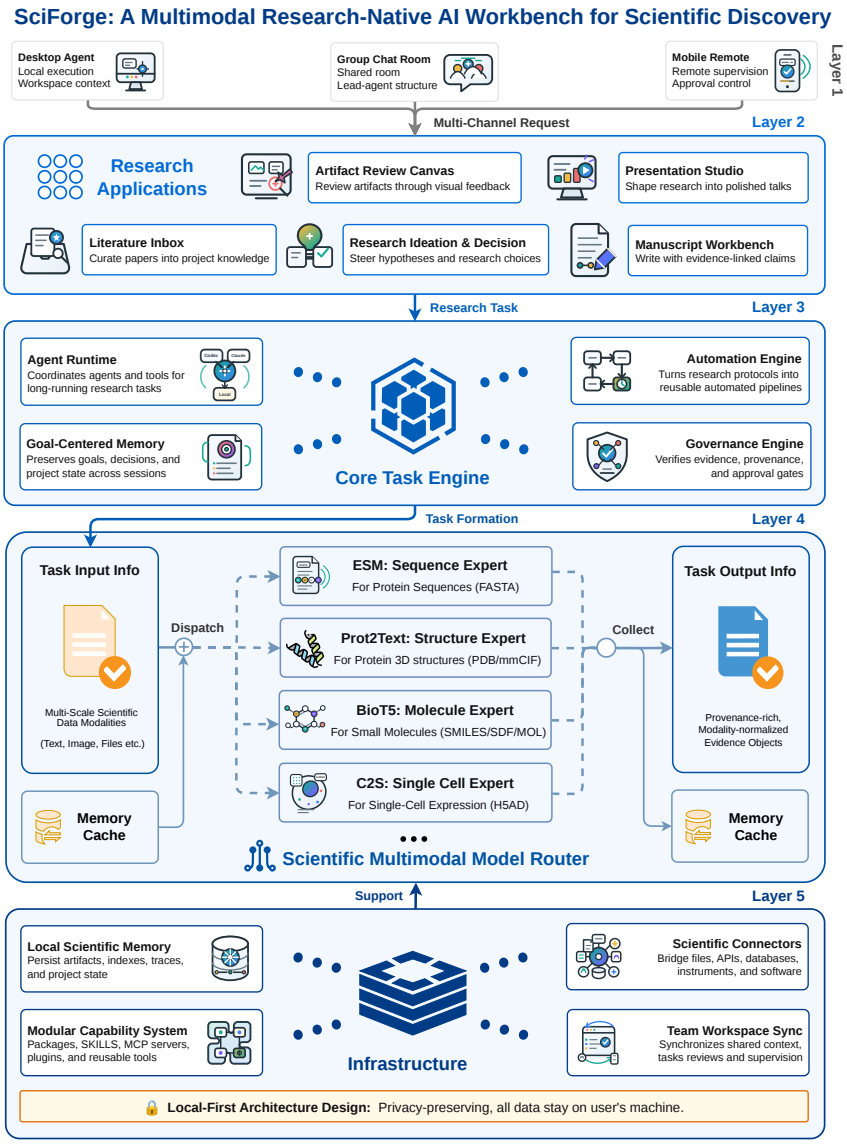

Fig. 1 SciForge system framework. The architecture organizes the workbench into user interaction and collaboration, research capability patterns, Core Engine, Scientific Model Router, and Infrastructure layers. User requests flow from desktop, group, and mobile surfaces into the Agent Runtime, which coordinates contextual research workflows, workflow automation, scientific modality routing, multimodal evidence generation, Evidence-DAG audit runs, and local-first infrastructure services.

or cloud upload. Researchers can edit code, inspect results, and iterate on experiments with the full power of local tooling. **Group coordination and mobile supervision** are implemented through group-chat integrations—Zulip, Discord, WeChat, and Feishu—enabling team discussion, shared review, cross-session handoff, and

lightweight monitoring/approval of long-running work. Team members can submit tasks remotely, supervise status, and coordinate through channel-based messaging; explicit annotation, task transfer, and approval of Project DAG decisions through chat remain illustrative until end-to-end integration evidence is available.

3.3 Research Capability Patterns (Layer 2)

Research capability patterns provide a unified entry point into scientific workflows, consolidating literature triage, data exploration, hypothesis exploration, protocol design, paper reproduction, scholarly writing, visual review, and presentation (PPT) refinement as contextual decision patterns within a single workbench rather than as separate standalone panels. Two complementary DAG substrates underpin all patterns. The **Evidence DAG** is thread-scoped: each agent thread grows its own claim–source–reasoning graph that records the evidentiary structure of that conversation's reasoning. The **Project DAG** is goal-scoped: it aggregates all auditable evidence across threads into a project-level graph that serves the overarching research goal, linking claims, audit findings, and decision records to the objectives they support. Together, thread-level Evidence DAGs provide fine-grained conversational provenance, while the goal-oriented Project DAG composes them into a unified, inspectable trail of the entire research effort. Audit runs record findings, and lightweight decision records capture explicit human decisions such as approvals, artifact reviews, and next actions. All background work—search, PDF parsing, data exploration, plotting, protocol drafting, paper reproduction pipelining, grant-narrative generation, PPT generation, and workflow execution—is delegated to the agent runtime and workers, keeping the primary workbench surface available for human judgment.

3.4 Core Engine (Layer 3)

The Core Engine plans and executes research work through four coordinated subsystems. The **Agent Runtime** uniformly supports multiple agent backends—Codex, Claude Code, and custom runtimes—allowing research groups to select the execution environment that best matches their scenario: Codex for local-first interactive coding with full workspace access, Claude Code for extended autonomous task chains, and custom runtimes for specialized scientific computing or institutional deployment requirements. Across all backends, the Runtime coordinates the main agent, delegated sub-agents, tool execution, workspace operations, MCP worker invocation, and long-running task recovery through a consistent API surface. The **Automation Engine** formalizes reusable pipelines with code, agent, tool, and approval nodes; scheduled execution and loop-style fixed protocol automation are treated as run modes rather than separate architectural modules. The **Goal-Centered Research Memory** stores scoped free-text records (scope, tags, confidence, timestamps, context) across sessions; typed goals and decision events belong to the Project DAG. The **Governance Engine** captures provenance, claim–evidence links, quality gates, approval boundaries, and internal Evidence DAG structures. Multimodal translation caches store scientific and visual observations by content hash, enabling repeated requests to reuse expert evidence without recomputing final conclusions.

Evidence DAG and Evidence Audit Runs. The Evidence-DAG sidecar builds a thread-scoped claim–evidence graph from completed agent turns. The runtime normalizes user messages, assistant/reasoning items, and successful tool results into a trace feed, then posts completed-turn deltas that grow the thread graph by default; explicit rebuilds are reserved for whole-thread reconstruction. The sidecar extracts source, reasoning, and claim nodes; verifies newly added support and contradiction edges when a Model Router judge is available; persists the graph as PROV-JSON; and runs a lightweight deterministic audit by default. Verification and automatic audit are advisory and fail open, so they do not block the agent's exploratory work. The Workbench Evidence DAG panel can also manually build and audit the current thread: the manual action backfills the active thread through the same trace feed, runs an audit over the current DAG, refreshes the embedded view, and reports a risk digest. Audit runs do not mutate the graph. They produce structured findings—for example ungrounded claims, unresolved contradictions, weak support, single-source dependencies, hidden shared-source support, load-bearing evidence, or low-credibility sources—which can later inform human decision records, workflow approval gates, or pre-publication commit checks.

3.5 Scientific Model Router (Layer 4)

The **Scientific Model Router** is a core architectural innovation of SciForge, acting as a universal model invocation plane that abstracts away provider heterogeneity while retaining the full expressiveness of each backend. Rather than forcing a lowest-common-denominator interface, the Router supports three request–response protocols—OpenAI Chat Completions, OpenAI Responses, and Anthropic Messages—selected per provider through a normalized endpoint format, allowing each model to be invoked through its native, most capable API surface.

Intelligent Auto-Routing. The Router includes a lightweight classifier (designed for low-latency) that inspects the user request and recent context to automatically select the appropriate reasoning effort (off, high, or max), reducing the need for researchers to manually tune reasoning budgets. The classifier is itself powered by a fast, low-cost model call, with a deterministic heuristic fallback that is designed so that routing decisions do not rely on a single point of failure.

Provider-Aware Optimization. For DeepSeek models, the Router performs proactive reachability probing, applies DeepSeek-native thinking controls (including automatic thinking toggle based on model capability), and computes real-time cache-aware cost estimation in both USD and CNY—surfacing token-level savings from prompt-cache hits and misses directly to the researcher. For all providers, it implements automatic retry with backoff, intelligent error classification (distinguishing transient failures from permanent configuration errors), and stream-idle timeout detection to prevent hung turns.

Output Robustness and Repair. The Router repairs malformed tool-call arguments from providers that emit non-conforming JSON—stripping Markdown fences, extracting balanced JSON objects, and recovering partial argument blocks—before the turn ever reaches the Agent Runtime. It also extracts embedded images from tool results (e.g., plots, diagrams) and re-encodes them as standard model attachments,

enabling multimodal reasoning chains that span scientific visualization outputs and subsequent model analysis.

Scientific Modality Routing. For scientific files, the Router detects modality-specific extensions and applies one of two handling paths. *Translate-then-reason* applies to three currently supported modalities: protein (.faa; .fasta/.fa with per-record conservative content classification) translated by Esm2Text, protein structures (.pdb, .cif, .mmcif) by Prot2Text, and small molecules (.smi, .smiles) by BioT5. C2S is a worker API contract, not an end-to-end file ingress route. Each translator output carries the modality, expert id, input reference, summary, and provenance metadata, allowing the main agent to reason over structured expert observations rather than opaque scientific encodings. VCF, BED, GFF, and MGF are unsupported and fail closed; raw content is not routed to the general reasoner.

3.6 Infrastructure (Layer 5)

The Infrastructure layer provides the durable substrate that anchors all research activity to the local workspace. **Local Scientific Memory** stores the substrate of research work: papers, datasets, parsed documents, scientific file manifests, embeddings, indexes, tool outputs, run metadata, and artifact caches. **Modular Capability System** packages computational functionality as Skills, MCP servers, Plugins, HTTP services, or worker processes—each independently versioned and upgradeable. **Scientific Connectors** normalize external resources such as papers, datasets, databases, APIs, instruments, notebooks, analysis scripts, reference managers, LIMS/ELN systems, and institutional storage into a unified access layer. **Team Workspace Sync** distributes selected state, summaries, review packets, approval requests, and mobile supervision signals across team members while keeping the underlying data anchored to the local project workspace.

4 Use Cases

This section uses three evidence-status labels consistently: *[Implemented]* for a product path exercised in SciForge, *[Validated Prototype]* for a bounded single-case or externally archived pilot with stated limitations, and *[Illustrative Future]* for an unexecuted target workflow. These cases are not separate product panels; they show what is implemented, what has only pilot evidence, and what remains a design target on the same modular substrate.

4.1 Agentic Research Sprint: From Question to Manuscript Package

Setting. *[Implemented]* A research group—led by a human PI—poses an open-ended biological question, *"Which molecular systems drive mammalian germ cells into meiosis?"*, and defines the research requirements and stage-gates. The PI then delegates execution to Codex, an AI coding agent that acts on the PI's behalf: Codex repeatedly invokes SciForge's agent-accessible services—literature search, candidate gene identification, structured hypothesis formulation, and parallel sub-agents for multi-omics

Fig. 2 Classification of AI-for-Life-Science scenarios represented by SciForge use cases. The labels distinguish an exercised product path (Implemented), bounded pilot evidence (Validated Prototype), and unexecuted design targets (Illustrative Future); they do not imply equivalent validation across cases.

Scientific scenario	**Representative workflow**	**Evidence status**
Literature-driven discovery	Agentic research sprint: meiosis initiation switch (scenario-01)	[**Implemented**]
Automated AI modeling	Protein–ligand binding, cellular trajectory, and materials-property prediction	[**Illustrative Future**]
Evidence-governed writing	Reviewer/rebuttal mode with a claim–evidence response matrix	[**Validated Prototype**]
Paper reproduction	MCFST spatial-transcriptomics guided reproduction	[**Validated Prototype**]
Multi-source integration	Cross-scale cell atlas spanning ECCITE-seq and external resources	[**Illustrative Future**]
Molecular discovery	AI-guided protein design for stability, specificity, or *de novo* binding	[**Illustrative Future**]
Lab automation	BGC-to-molecule closed-loop discovery for marine fungi	[**Illustrative Future**]

evidence synthesis, AlphaFold3 structure-based computational assessment, and iterative methodology hardening—within a multi-day, PI-controlled agentic loop (Fig. 3). We emphasize that Codex serves as the PI's AI delegate and does *not* replace SciForge; SciForge remains the research workbench whose modular services (search, routing, workflows, plotting, writing) the agent consumes. Selected outputs are reviewed through evidence summaries, reviewer-style self-audits, and human decisions; the PI tightens evidence boundaries and repairs tooling as needed. After 132 stages and 197 Git-tracked commits, the repository[1] records the control loop as a local workspace log: hypotheses, evidence ingestion, structural analysis, manuscript integration, and reviewer-style self-audits.

Discovery. The systematic agentic pipeline yielded a candidate-gene atlas of 23 genes across 12 molecular classes, with claims traceable to specific evidence sources (Fig. 4). Central to the discovery was the DCT-M3 causal-module triangulation framework, which decomposes the meiosis initiation switch into four regulatory layers: (1) permissive licensing—chromatin opening and epigenetic priming via pioneer transcription factors; (2) brake release—degradation of meiotic inhibitors through ubiquitin-proteasome pathways; (3) trigger execution—SPO11-mediated programmed double-strand breaks and synaptonemal complex assembly; and (4) chromatin/RNA competence—transcriptional activation, alternative splicing, and 3D genome reorganization. Multi-omics evidence (transcriptomics, epigenomics, proteomics) was cross-validated against AlphaFold3 structure-based computational assessments and systematic literature triangulation to ground each candidate in causal evidence. This scenario demonstrates that SciForge can sustain long-horizon, evidence-constrained research—not by auto-generating a paper, but by producing an inspectable, auditable research package whose reasoning and evidence trail remain open to human scrutiny.

[1] https://github.com/AGI4Sci/scenario-01-research-sprint (archive currently unavailable; the URL returned 404 at time of writing)

Informal Post-Hoc Literature Mapping. A post-hoc review of the 23-gene atlas against the published meiosis literature found that 22 of the 23 genes have known meiosis associations; MAPK fell outside established meiosis paradigms. This retrospective mapping was conducted informally and was not guided by a pre-registered protocol. Formal expert adjudication of gene-level relevance, along with inter-annotator agreement metrics and a pre-registered evaluation protocol, remains future work.

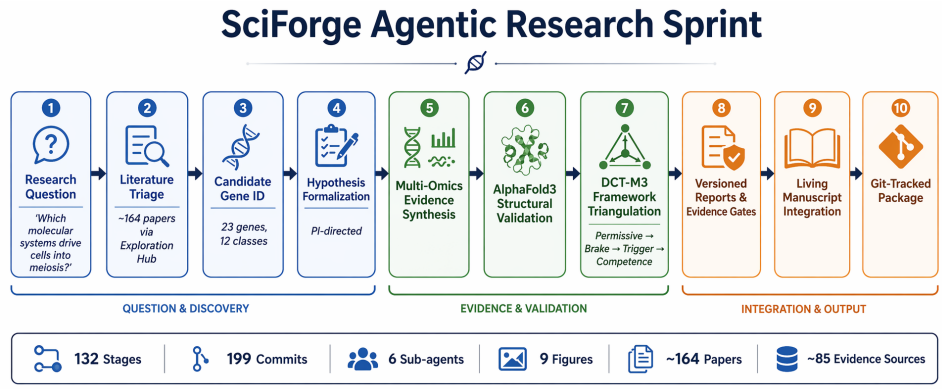

Fig. 3 Automated research pipeline executed during the SciForge scenario-01 research sprint. The 132-stage, 197-commit agentic loop progresses from a biological research question through literature mining (DCT-M3), candidate gene curation, AlphaFold3 structural prediction, evidence triangulation, manuscript synthesis, and submission packaging. Quantitative metrics: 132 research stages, 197 Git commits, 6 parallel child-agent runs, 9 generated figures, ∼164 papers reviewed, and ∼85 evidence nodes produced. Selected artifacts and thread evidence are linked to versioned records, evidence summaries, and audit findings where captured.

4.2 AI4AI: Automated Modeling for Scientific Discovery

[Illustrative Future] This illustrative target workflow describes how SciForge could support AI-for-AI (AI4AI) loops in which agents propose, train, evaluate, and iterate on machine learning models for scientific tasks. A researcher specifies a modeling objective—such as predicting protein–ligand binding affinity, reconstructing cellular trajectories, or forecasting materials properties—together with a dataset and evaluation criteria. SciForge's agent runtime can then compose architecture search, hyperparameter optimization, training with checkpointed runs, benchmark evaluation against baselines, and failure analysis with diagnostic reports. Model artifacts, training logs, evaluation metrics, and experimental configurations can be linked to evidence provenance and audit findings when they are captured by the runtime. This target workflow illustrates how SciForge's substrate could treat the model itself as a first-class research artifact.

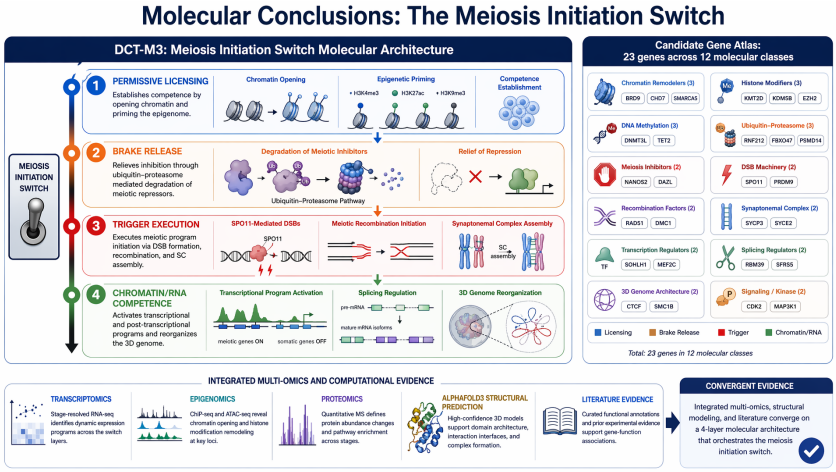

Fig. 4 Key biological conclusions reported by the SciForge scenario-01 agentic research sprint. The DCT-M3 causal-module triangulation framework decomposes the meiosis initiation switch into four regulatory layers: (1) permissive licensing—chromatin opening and epigenetic priming; (2) brake release—degradation of meiotic inhibitors via ubiquitin-proteasome pathways; (3) trigger execution—SPO11-mediated double-strand breaks and synaptonemal complex assembly; (4) chromatin/RNA competence—transcriptional activation, splicing regulation, and 3D genome reorganization. The sprint reported a candidate-gene atlas of 23 genes across 12 molecular classes, supported by multi-omics evidence, AlphaFold3 structure-based computational assessment, and literature triangulation; formal expert adjudication remains future work.

4.3 Reviewer / Rebuttal Mode: Evidence-Governed Peer Review and Response

Scenario. *[Validated Prototype]* To exercise SciForge's reviewer/rebuttal capability, we conducted a structured six-stage sprint against the manuscript *"Benchmarking Virtual Cell Models for In-the-Wild Perturbation Response"* (arXiv:2604.27646v1, VCBench) [18]. This is a one-manuscript SciForge-assisted first pass; real reviewer comments, follow-up analyses, and manuscript revisions are not yet completed. The full sprint log, outputs, and SciForge GUI trace are archived in a dedicated repository.[2]

Stage 1—Intake. The manuscript PDF and LaTeX source were frozen and ingested. Figures, tables, datasets, and code references were catalogued as evidence objects.

Stage 2—Claim Extraction. Contribution claims, result claims, limitation claims, and practical-guidance claims were extracted and assigned to paper sections and evidence objects.

Stage 3—Evidence Linking. Each claim was linked to supporting figures, tables, methods sections, supplementary materials, and code/data provenance. Missing links and evidence gaps were flagged.

Stage 4—Fragility Audit. Claims were classified into one of five categories: `supported` (evidence present, wording matches scope), `fragile` (evidence exists

[2] https://github.com/maoxinjie/scenario-05-reviewer-rebuttal-vcbench

but depends on narrow conditions), `unsupported` (assertion exceeds current evidence), `needs_narrowing` (claim can stand with tighter scope), or `needs_experiment` (requires new analysis or validation). Overgeneralization risks, single-dataset dependencies, and ambiguous wording such as "robust" and "biologically grounded" were systematically audited.

Stage 5—Reviewer Decomposition. Ten anticipated reviewer concerns were decomposed into atomic comments and mapped to specific claims, evidence requirements, and response paths (manuscript edit, supplementary analysis, or wording revision).

Stage 6—Rebuttal and Revision Planning. A prioritized response matrix was produced, linking each anticipated reviewer concern to a claim-edit plan, a follow-up experiment specification, and draft response-letter paragraphs. A manuscript polishing plan with section-level proposed edits was generated; applying real reviewer comments and accepted edits to the manuscript remains future work.

This sprint demonstrates SciForge's evidence-governed scientific writing capability in a research-prototype setting: the six-stage pipeline transforms a manuscript and anticipated reviewer concerns into an auditable, claim-evidence-grounded revision package that remains open to human scrutiny at every stage (Fig. 5). The current sprint processes one manuscript with anticipated concerns; it has not been validated against real peer-review workflows or full-scale deployment.

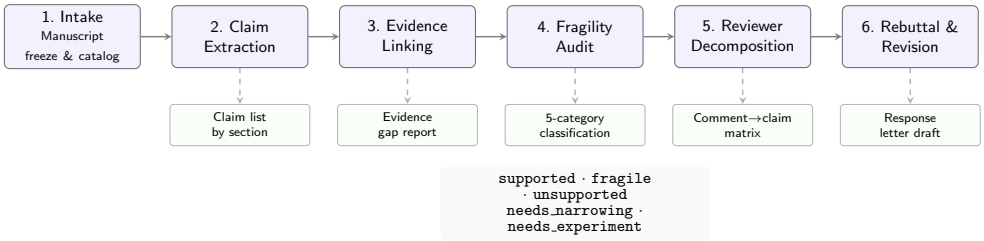

Fig. 5 Six-stage Reviewer/Rebuttal workflow validated against the VCBench manuscript (arXiv:2604.27646v1). The pipeline transforms a manuscript and reviewer feedback into a claim-evidence-grounded revision package with a response-letter draft and a decision log.

4.4 Guided Paper Reproduction: MCFST Spatial Transcriptomics

Setting. A researcher aims to reproduce the MCFST method [19]—a spatial domain identification approach based on multi-view graph convolutional networks and graph fusion—using the Human Breast Cancer Visium spatial transcriptomics dataset. The goal is to independently re-implement the method, reproduce the reported ARI (Adjusted Rand Index) metric, and generate publication-quality verification artifacts.

Workflow. The reproduction process follows a structured AI4AI pipeline orchestrated within SciForge:

1. **Paper Intake.** The researcher provides the MCFST manuscript and associated supplementary materials to SciForge. The system parses the paper, extracting the model architecture description, training hyperparameters, evaluation protocol, and dataset details into structured evidence records.
2. **Code Generation and Adaptation.** Based on the extracted methodological specifications, SciForge generates a standalone Python implementation comprising: a multi-view graph construction module (`process.py`), a graph autoencoder with adversarial regularization (`gae_v4.py`), a training and evaluation loop (`main_v4.py`), and a visualization script (`plot_results.py`). The implementation targets CPU execution on Apple Silicon, adapting the original GPU-oriented design to the available hardware.
3. **Data Preprocessing.** The Visium spatial transcriptomics dataset (3,798 spots, 20 spatial domains) is preprocessed through PCA dimensionality reduction to 3,000 features. Four distinct graph views are constructed from the spatial neighborhood and gene expression similarity matrices, with mutual information-based edge pruning applied to each view.
4. **Reproduction Execution.** The training pipeline runs for 130 epochs across multiple random seeds. Each run produces clustering predictions saved as structured NumPy arrays, enabling full traceability and independent verification.
5. **Automated Verification.** A dedicated verification script (`verify.py`) recomputes the ARI score for every saved prediction against the ground-truth labels and compares the best result against the paper's reported ARI of 0.693. The script generates a structured JSON verification report recording the verdict, all individual ARI scores, dataset statistics, and hardware configuration.
6. **Artifact Generation.** The pipeline produces publication-quality figures—spatial domain maps, ARI comparison charts across runs, and domain agreement heatmaps—alongside a summary metrics report.

Outcome. *[Validated Prototype]* The reproduction achieved a best ARI of **0.7007**, with an all-20 mean of 0.4617 ($\pm$0.1964) across 20 predictions and a selected-5 mean of 0.6317 ($\pm$0.0870). The 0.05 success threshold was applied post-hoc and the selection rule for the five runs is not pre-specified; `verify.py` reports `n_runs=5`, conflicting with the 20 total predictions. The result is therefore reported as a prototype demonstration, not a formal reproduction. Formal verification would require a pre-registered contract and third-party re-execution. All generated code, preprocessed data, prediction outputs, and result figures are version-controlled and available at https://github.com/Winshion/sciforge-ai4ai-spacial-trans.

Significance. This case demonstrates that SciForge can support a prototype computational paper reproduction pipeline: from parsing the methodological description to generating executable code, running the experiment, and producing verification documentation. The structured verification contract requires revision (pre-registered

selection rule, reconciled run counts) before it can support independent third-party audit; in its current form it illustrates feasibility rather than closing the reproducibility loop.

4.5 Cross-Scale Cell Atlas from Multi-Database Integration

[Illustrative Future] We describe SciForge's data-integration capability through a planned cross-scale cell atlas construction workflow anchored on the Papalexi/Satija 2021 ECCITE-seq dataset (GEO GSE153056). This scenario spans a **six-layer biology stack**: **L0** perturbation (CRISPR guide/target gene, ECCITE-seq), **L1** target annotation (UniProt, external curated), **L2** pathway context (Reactome/GO, external computed), **L3** RNA response (scRNA guide-versus-control differential expression, ECCITE-seq), **L4** protein-level mechanism response (ADT/CITE-seq, ECCITE-seq), and **L5** endpoint phenotype (DepMap/Achilles gene effect in THP-1 cells, external computed). Only L0, L3, and L4 are traceable to ECCITE-seq measurements; L1 (UniProt), L2 (Reactome/GO), and L5 (DepMap) are external curated or computed resources. The heatmap is a schematic synthetic matrix of 16 rows (12 targets and 4 controls), not 15 real target measurements. No `case45_gate.py`, phase JSON, atlas/provenance manifest, gate outputs, or Evidence Snapshot artifacts exist for this scenario.

The planned workflow, illustrated in Figure 6, is specified for SciForge's remote execution infrastructure: a researcher would issue high-level instructions from the local frontend while an agent runtime on the PJLab compute cluster performs data retrieval, processing, and quality control through an eight-phase pipeline (P0–P7). This workflow has not been executed; the phases are design specifications. **P0** performs preflight checks—Git credentials, workspace audit, and frame manifest. **P1** inventories GEO supplementary files and confirms matrix availability. **P2** selects 20–50 guide/gene targets across JAK/STAT, IFN, NF-κB, checkpoint, and control classes. **P3** generates extraction scripts for scRNA and ADT counts with UniProt annotation. **P4** computes guide-level L3 RNA and L4 ADT response with Reactome/GO enrichment. **P5** aligns L0–L4 records with DepMap/Achilles L5 phenotypes, classifying each guide as a strong, moderate, or weak dependency. **P6** assembles the L0–L5 dataset with provenance manifests and schema validation. **P7** produces automated exploratory outputs including coverage analysis and cross-scale linkage visualization.

The design specifies a **script-first** evidence contract: the agent would generate standalone Python or R scripts under a workspace-relative `scripts/` directory, execute them via a remote executor, and record script paths and commands in per-phase status JSON files. Interactive inline commands or ad-hoc shell pipelines would not be accepted as formal phase evidence. Automated gate tools (for example, a proposed `case45_gate.py`) would validate phase outputs. No such gate tool, per-phase JSON, or executed contract currently exists.

When executed, SciForge's Evidence DAG would be designed to capture provenance metadata—public source name and accession, retrieval timestamp, local derivative path, transformation step, and whether the layer represents native measurement, external curation, or computed fixture—as records flow through the pipeline. The resulting L0–L5 atlas and its provenance report could then feed into the Exploration

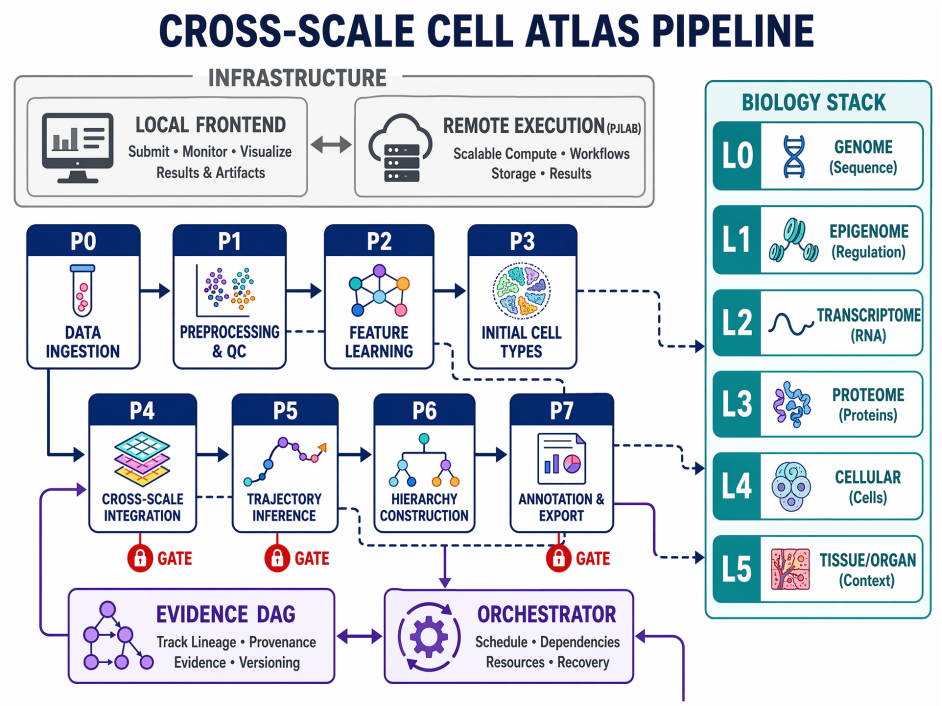

Fig. 6 Cross-Scale Cell Atlas Pipeline *[Illustrative Future]*. The planned workflow spans eight phases (P0–P7) from local frontend instruction through data retrieval and multi-layer processing to an Evidence DAG–anchored output. Dashed arrows connect pipeline phases to the corresponding layers of the six-layer biology stack (L0–L5). Gate validation and evidence contracts are specified but not yet executed; no gate outputs or provenance manifests exist.

Hub for hypothesis generation or into AI4AI modeling workflows. The current description is a design sketch; no executed pipeline, provenance manifest, or analysis-ready dataset exists.

4.6 AI-Guided Protein Design

[Illustrative Future] In a target protein-design workflow, a researcher specifies a target function—such as enhanced thermostability, altered substrate specificity, or *de novo* binding to a target epitope—together with a starting protein structure or sequence family. SciForge could compose sequence–structure co-analysis via the Scientific Model Router, variant generation with evolutionary or inverse-folding models, structural and energetic evaluation, and multi-objective candidate ranking. Candidate records can link sequences, predicted structures, computational evidence, design rationales, and audit findings when those artifacts are captured by the Evidence DAG and decision-record substrate.

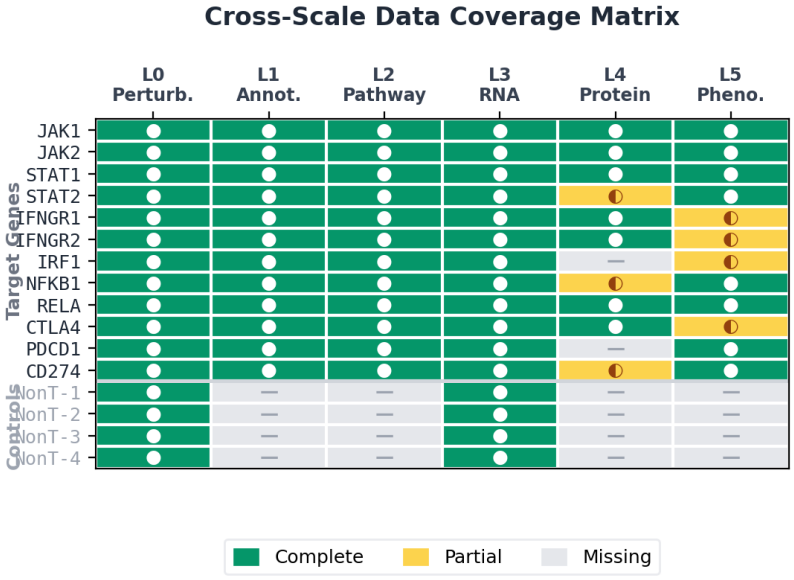

Fig. 7 Cross-Scale Data Coverage Matrix *[Illustrative Future]*. Schematic heatmap of 16 rows (12 targets and 4 controls) across six biological layers (L0–L5). Coverage scores: 0 = missing, 1 = partial, 2 = complete. The matrix is a hard-coded synthetic illustration; no real data processing or gate validation has been executed for this scenario.

4.7 Computer Use: BGC-to-Molecule Closed-Loop Discovery for Marine Fungal Dark Matter

[Illustrative Future] Marine fungi represent a vast reservoir of biosynthetic dark matter—uncharacterized biosynthetic gene clusters (BGCs) whose metabolic products remain unknown. In a target computer-use workflow, a researcher provides one or more fungal genome sequences, and SciForge's Computer Use harness coordinates local software and web-submission portals. On the genome side, an agent could submit jobs to antiSMASH for BGC prediction, query MIBiG for known-cluster comparison, and run BiG-SCAPE to construct gene cluster family similarity networks. On the metabolomics side, it could ingest MS/MS spectra, feature tables, and annotation tables, routing data through SIRIUS/CANOPUS and cross-referencing against GNPS molecular networks. The same Evidence DAG and audit layer can then help inspect whether candidate BGC, MS/MS evidence, BGC–metabolite links, result feedback, and experiment design records are sufficiently grounded before downstream decisions.

5 Evaluation

SciForge is designed as a research infrastructure platform, not as a single-task algorithm whose performance is measured by a scalar metric. We therefore frame evaluation around four questions that are relevant for an auditable, multimodal research

workbench, and identify what the current codebase and public scenario artifacts can support versus what requires future systematic evaluation.

5.1 Evaluation Questions and Criteria

1. **Modality Routing Coverage.** Does the Scientific Model Router correctly detect modality, select the configured translator, and attach provenance-annotated observations? *Current evidence:* three translator paths (Esm2Text, Prot2Text, BioT5) are implemented with unit-level integration in the codebase; C2S is a worker API contract, not a file ingress route; VCF/BED/GFF/MGF are unsupported and fail closed. *Future:* systematic coverage benchmarks across standard bioinformatics file corpora, false-positive/negative routing rates, and comparison against manual modality annotation.
2. **Provenance Completeness.** Does the Evidence DAG capture artifact identity, source anchors, software versions, parameters, and execution traces? *Current evidence:* PROV-JSON schemas and audit-sidecar contracts are present in the codebase; scenario-01, MCFST, and reviewer/rebuttal repositories archive selected workflow artifacts. Commit count alone does not constitute provenance or scientific validity evidence. *Future:* completeness audits against a ground-truth provenance corpus, measurement of missing-link rates under long-running agentic loops, and comparison against manual lab-notebook logging.
3. **Reproducibility.** Can a third party re-execute a recorded workflow from versioned artifacts and scripts? *Current evidence:* the cell-atlas pipeline is an illustrative specification (no execution artifacts are available); scenario-01 and MCFST repositories archive outputs but have not undergone third-party re-execution. *Future:* third-party re-execution trials with environment recreation, runtime variability measurement, and comparison against containerized workflow systems.
4. **Audit and Decision Governance.** Does the audit layer flag fragile claims, and does the release gate correctly distinguish blocking from non-blocking findings? *Current evidence:* lightweight decision-record contracts and the five-category fragility taxonomy are defined; the reviewer/rebuttal sprint exercises these on a single manuscript as a prototype demonstration, not a validated deployment. *Future:* controlled injection studies with known-weak and known-strong claims, measurement

of false-positive and false-negative audit rates, and user studies on decision-making efficiency under audit guidance.

5.2 Current Evidence Scope and Future Evaluation

The four evaluation criteria above are currently supported by (a) the code-level contracts, schemas, and CI-visible tests present in the SciForge repository; (b) the scenario-01 research sprint repository (archive currently unavailable); (c) the MCFST paper reproduction repository; and (d) the reviewer/rebuttal sprint repository. Together these artifacts constitute existence proofs that selected architectural components are instantiated in running code—they do *not* constitute systematic quantitative evaluation. Existence proofs demonstrate selected components and demonstrations; they are not a substitute for systematic evaluation.

The following evaluation categories are explicitly identified as **future work**: (i) baseline comparisons against alternative scientific workbench systems or manual workflows; (ii) ablation studies that isolate the contribution of individual components (e.g., translator routing, Evidence DAG, audit layer); (iii) inter-annotator agreement or expert-consensus studies on audit findings and evidence quality; (iv) time-to-completion or researcher-efficiency studies under controlled protocols; (v) scalability benchmarks under long-running multi-day agentic loops. We encourage the community to contribute protocols, benchmark datasets, and reproducibility studies that exercise these dimensions on the SciForge platform.

6 Discussion

6.1 Design Trade-offs

SciForge deliberately favors local control, inspectable state, and modular capability boundaries over a single fully managed cloud service. This improves privacy, deployment flexibility, and laboratory control, but it also requires careful configuration of local models, worker services, scientific connectors, and storage. The same trade-off appears in the Scientific Model Router: domain expert translation improves modality handling and evidence quality, but expert coverage depends on the configured translators and available compute.

The governance layer introduces another trade-off. Recording provenance, claim–evidence links, audit findings, approvals, and artifact review history can make scientific work more auditable, but excessive synchronous gating would slow agent exploration and burden users. SciForge therefore builds the Evidence DAG and runs lightweight audits after completed work or on explicit user request. The Project DAG exposes candidate/certified release records and blocking gate semantics; only product actions explicitly integrated with that API are currently gated.

6.2 Limitations and Future Work

The current implementation has several concrete limitations. **Ingress scope:** end-to-end file ingress is restricted to three modality classes (protein, protein_structure,

molecule); C2S is a worker API contract, not a file ingress route; VCF, BED, GFF, and MGF are unsupported and fail closed. **Scientific validation:** translator observations are evidence candidates, not verified scientific facts; no systematic validation against ground-truth annotations has been performed. **Memory model:** research memory stores scoped free-text records rather than a typed scientific state store; typed goals and decisions belong to the Project DAG. **Evidence completeness:** the Evidence DAG schema can represent complete provenance but depends on runtime capture; software, parameters, environment, log/output, and seed are not automatically populated. **IM governance:** IM-based coordination is partially integrated; annotation, task transfer, and DAG-decision approval through chat are not end-to-end validated. **Evaluation gaps:** no baselines, ablation studies, user studies, or third-party reproductions have been conducted. **MCFST contract:** the verification contract has unresolved conflicts (pre-selection rule, run-count mismatch) and the 0.05 threshold is post-hoc; formal reproduction requires revision and third-party re-execution. **Cell-atlas provenance:** the cell-atlas pipeline is a specification without execution artifacts; the heatmap is schematic, and `case45_gate.py` has not been exercised. Future work includes expanding scientific modality support to crystal structures, mass spectrometry, cryo-EM, and additional omics formats; implementing baselines and systematic evaluations; strengthening IM governance integration; improving provenance capture automation; and developing pre-registered verification contracts with third-party re-execution protocols.

7 Conclusion

SciForge presents a research-native AI workbench that brings together local execution, scientific multimodal routing, agent orchestration, workflow automation, research memory, and human-governed review surfaces. Its three design pillars—translate-then-reason for multimodal input, evidence governance for auditable traceability, and goal-scoped scientific decision governance—are designed so that supported native scientific objects become inspectable evidence candidates, captured claims can be linked to provenance records, and release decisions remain reviewable. By combining a local-first workspace with modular worker services and contextual research capability patterns, SciForge aims to serve as an auditable, collaborative agent operating environment for everyday scientific discovery.

Appendix A Capability Differentiation

Table A1 Capability positioning based on reviewed public materials. Typical-approach entries summarize common documented emphases and do not prove absence of unlisted features.

Capability	Function	Typical approaches	SciForge differentiation
Model Router	Unified provider and model management	Single-provider lock-in or manual switching	Routes across OpenAI, Anthropic, DeepSeek, and local models with fallback and localizable deployment
Scientific Pipeline	Convert scientific objects into agent-readable evidence	General LLMs receive raw scientific text or require manual preprocessing	Translate-then-reason with three modality experts (protein, structure, molecule); C2S is a worker contract; unsupported formats fail closed
Agent Runtime	Coordinate agents, tools, commands, and long tasks	Single-session chat execution with limited delegation	Workspace-aware runtime with sub-agent delegation, MCP workers, plan recovery, and task continuation
Workflow Engine	Automate repeatable research protocols	Scripts, cron jobs, and manual runbooks	Code, agent, tool, schedule, event, and approval nodes under one research workflow model
Research Memory	Preserve project state across sessions	Chat history or document RAG	Scoped free-text records (scope, tags, confidence, timestamps, context); typed goals and decisions belong to Project DAG
Evidence Governance	Connect claims, evidence, audit findings, and approvals	Outputs remain detached from data, scripts, and review history	Thread-scoped Evidence DAGs with provenance capture when runtime supplies evidenceLineage; audit runs, risk digests, quality gates, and approval boundaries
Research Capability Patterns	Expose human judgment points	Generic chat and file views	Unified workbench patterns backed by Evidence DAGs, audit findings, and lightweight decision records for approvals, artifact review, and next actions

2 Resource

The SciForge codebase, including the Evidence-DAG audit worker, lightweight decision-record contracts, agentic skills, and scientific model router, is open source and available at (the scenario-01 research-sprint gallery URL currently returns 404):

https://github.com/AGI4Sci/SciForge

The MCFST paper reproduction artifacts—including generated code, preprocessed data, prediction outputs, and result figures—are available at:

https://github.com/Winshion/sciforge-ai4ai-spacial-trans

Researchers are encouraged to explore both repositories, contribute new skills, workflows, audit rules, and research decision patterns, and develop third-party re-execution protocols for the workflows described in this paper.

References

[1] Allen Institute for AI (2025) Ai2 ScholarQA. Available at https://allenai.org/blog/ai2-scholarqa (accessed 4 July 2026)

[2] Anthropic (2024) Introducing the Model Context Protocol. Available at https://www.anthropic.com/news/model-context-protocol (accessed 4 July 2026)

[3] Anthropic (2026) Claude Science: An AI Workbench for Scientists. Available at https://www.anthropic.com/news/claude-science-ai-workbench (accessed 4 July 2026)

[4] Asai A, et al (2024) OpenScholar: Synthesizing Scientific Literature with Retrieval-Augmented Language Models. Preprint at https://arxiv.org/abs/2411.14199

[5] ClaudePrism Contributors (2026) ClaudePrism: Desktop Workspace for Scientific Writing. Available at https://github.com/delibae/claude-prism (accessed 4 July 2026)

[6] Consensus (2026) Consensus Research Agent. Available at https://help.consensus.app/en/articles/12641232-research-agent (accessed 4 July 2026)

[7] Elicit (2026) Elicit Research Agent. Available at https://elicit.com/ (accessed 4 July 2026)

[8] FutureHouse (2024) PaperQA2: Retrieval-Augmented Generative Agent for Scientific Research. Preprint at https://arxiv.org/abs/2409.13740

[9] FutureHouse (2025) Launching FutureHouse Platform AI Agents. Available at https://www.futurehouse.org/news/launching-futurehouse-platform-ai-agents (accessed 4 July 2026)

[10] Google Research (2025) Accelerating Scientific Breakthroughs with an AI Co-Scientist. Available at https://research.google/blog/accelerating-scientific-breakthroughs-with-an-ai-co-scientist/ (accessed 4 July 2026)

[11] Lu C, Lu C, Lange RT, et al (2024) The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery. Preprint at https://arxiv.org/abs/2408.06292

[12] OmicOS (2026) OmicOS Overview. Available at https://docs.omicos.cn/zh/part1/01-overview.html (accessed 4 July 2026)

[13] OmicOS (2026) Starting OmicOS CLI. Available at https://docs.omicos.cn/zh/part1/08-starting-cli.html (accessed 4 July 2026)

[14] OmicOS (2026) Starting OmicOS Serve. Available at https://docs.omicos.cn/zh/part1/07-starting-serve.html (accessed 4 July 2026)

[15] OmicsClaw Contributors (2026) OmicsClaw: Local-First Multi-Omics Research Assistant. Available at https://github.com/TianGzlab/OmicsClaw (accessed 4 July 2026)

[16] Schmidgall S, et al (2025) Agent Laboratory: Using LLM Agents as Research Assistants. Preprint at https://arxiv.org/abs/2501.04227

[17] Swarup Lab (2026) Operon: AI-Powered IDE for Bioinformatics. Available at https://github.com/swaruplab/operon (accessed 4 July 2026)

[18] Yang K, et al (2026) Benchmarking virtual cell models for in-the-wild perturbation response. arXiv preprint arXiv:260427646v1 ArXiv:2604.27646v1

[19] Zhang Z, Duan H, Gao X (2026) MCFST: Spatial domain identification method based on multi-view graph convolutional network and graph fusion network. Bioinformatics https://doi.org/10.1093/bioinformatics/btag469